WARNING ASP.NET Core doesn't automatically validate antiforgery tokens if you're using MVC controllers with Views. You must make sure to mark all vulnerable methods with `[ValidateAntiForgeryToken]` attributes instead, as described in the "Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core" documentation: http://mng.bz/QPPv. Note that if you're not using cookies for authentication, you are not vulnerable to CSRF attacks: CSRF attacks arise from attackers exploiting the fact that browsers automatically attach cookies to requests. No cookies, no problem!

Generally, you need to use antiforgery tokens only for `POST`, `DELETE`, and other dangerous request types that are used for modifying state. `GET` requests shouldn't be used for this purpose, so the framework doesn't require valid antiforgery tokens to call them. Razor Pages validates antiforgery tokens for dangerous verbs like `POST` and ignores safe verbs like `GET`. As long as you create your app following this pattern (and you should!), the framework does the right thing to keep you safe.

If you need to explicitly ignore antiforgery tokens on a Razor Page for some reason, you can disable the validation by applying the `[IgnoreAntiforgeryToken]` attribute to a Razor Page's `PageModel`. This bypasses the framework protections for those cases when you're doing something that you know is safe and doesn't need protecting, but in most cases it's better to play it safe and validate.

CSRF attacks can be a tricky thing to get your head around from a technical point of view, but for the most part everything should work without much effort on your part. Razor adds antiforgery tokens to your forms, and the Razor Pages framework takes care of validation for you.